Model improvement on Stampede3

Build a reproducible adapter program around official Gemma 4 weights, with **Gemma 4 12B Unified as the first BF16 LoRA candidate**, Gemma 4 26B-A4B as the parallel MoE candidate, and Gemma 4 31B as a larger reference and possible teacher. Concentrate the first adapter on source-grounded work, then develop correction handling and tool-use specializations. Keep retrieval, memory, and access control independently testable so that an apparent improvement can be attributed to the right component.

The opportunity is to make eligible models substantially better at a defined workflow. Matching the strongest unrestricted general model is an experimental question, not a promised outcome of LoRA. The useful deliverable is an adapter that completes more tasks correctly at an acceptable cost, with reproducible evidence and a distribution format that independently operated peers can use.

The planning scope includes the previously established whole-Stampede3 reservation. A bounded training probe establishes a working execution path; it does not reduce that reservation to a four-node project. Hardware compatibility, data preparation, and baseline evaluation should proceed as parallel workstreams, followed by parallel controlled comparisons.

Hardware and evidence

TACC's operational guide lists **20 PVC nodes, four Intel Data Center GPU Max 1550 devices per node**, with two 62 GB memory tiles per device and 124 GB combined. Each PVC node has 1 TB host RAM. The guide also lists 24 four-H100 nodes. Its inventory supersedes the older system overview's 10-node Intel GPU description for planning, subject to actual reservation inventory.[1][2]

The important unknown is how the selected driver and runtime expose those tiles. A process may not have one simple allocation space equal to the sum of every tile on a node. Record physical devices, logical devices, tile affinity, and allocatable memory before deciding whether a checkpoint fits. Neither 496 GB across four physical devices nor host RAM constitutes a single automatically available training allocation.

Claim	Evidence level	Consequence
Stampede3 has Intel PVC GPUs	Documented by TACC	Design and benchmark an XPU training path.
Gemma 4 inference works on Intel hardware	Documented by Intel; Stampede3 inference also reported by the project	Preserve the successful inference recipe as a reference.
Native PyTorch can train on PVC	Documented upstream, including training examples	BF16 training has a credible foundation.
bitsandbytes supports XPU four-bit LoRA	Documented in its release support matrix	QLoRA is a candidate, rather than a CUDA-only assumption.
Gemma 4 + PEFT + the installed Stampede3 stack can train and resume	Not yet established by a retained training receipt	Require actual backward, update, save, reload, and resume tests.
Gemma expert quantization recipes transfer from CUDA to PVC	Not established; one relevant implementation explicitly requires CUDA	Do not make those recipes prerequisites for the first useful adapter.

Intel documents Gemma 4 attention and MoE support in Transformers/vLLM, but this is deployment evidence rather than a Stampede3 LoRA training benchmark. PyTorch documents native XPU training on Max Series

devices. These establish reasons to attempt the experiment, not its throughput, stability, or final memory footprint.[3][4]

Model selection and eligibility

Apply the project's American-model requirement to the entire new model pipeline: students, teachers, judges, embeddings, rerankers, transcription, and OCR. The working rule follows the originating/publishing organization; Google Gemma is explicitly accepted. This is a project requirement, not a claim that TACC generally prohibits other models. Hosting location and model origin remain separate decisions.

Use an artifact manifest containing original publisher, base model, derivative ancestry, repository revision, file hashes, license, tokenizer/template revision, and intended input destinations. A US host or a US republisher does not by itself establish eligibility for an otherwise disallowed base. Newly generated training examples must also record their teacher; do not import unreviewed synthetic mixtures or use disallowed teachers to evade the restriction.

Google's current card covers E2B, E4B, 12B Unified, 26B-A4B, and 31B, and identifies Gemma 4's license as Apache 2.0. The E-model names describe effective size, not all stored parameters. The MoE card describes approximately 3.8 billion active parameters, 128 routed experts with eight selected, and one shared expert.[5]

The following weight-only estimates use exact parameter totals reported by the official Hugging Face repositories on September 13, 2026. They include the checkpoint's reported tensors, which explains differences from rounded language-backbone counts. BF16 bytes are `2 × parameters`; GiB divides bytes by `2^30`. They exclude activations, gradients, optimizer state, temporary buffers, and allocator overhead. The companion model inventory records the exact revisions and formulas.[6]

Official instruction checkpoint	Reported stored parameters	BF16 weights, decimal GB / GiB	Recommended role
`google/gemma-4-E4B-it`	7,996,156,490	15.99 / 14.89	Smaller peer-deployment target; fallback training candidate.
`google/gemma-4-12B-it`	11,959,730,224	23.92 / 22.28	First dense BF16 LoRA candidate.
`google/gemma-4-26B-A4B-it`	25,805,936,206	51.61 / 48.07	Parallel BF16 attention-adapter experiment; efficiency candidate.
`google/gemma-4-31B-it`	31,273,088,876	62.55 / 58.25	Larger baseline, candidate teacher, later dense adapter.

The 12B recommendation is an engineering judgment: it offers considerably more weight headroom than 26B/31B without requiring expert quantization. It uses `Gemma4UnifiedForConditionalGeneration`, whereas the other listed checkpoints use `Gemma4ForConditionalGeneration`. Unified has a different multimodal input architecture, so use its matching processor/class and verify text-only training explicitly.[7] If that integration fails its short probe while E4B succeeds, continue useful work on E4B and preserve the Unified failure as a separate issue.

The 31B checkpoint's weights alone approach an exposed 62 GB tile's capacity. Do not promise one-tile BF16 training. A successfully exposed whole physical GPU, validated sharding, or independently verified dense QLoRA can provide alternative placements. The 26B model also needs every resident expert's weights even when only some experts participate in a particular token's computation.

Stanley's HauhauCS Balanced recommendation should remain a comparison candidate. Its card distributes GGUF variants and claims reduced refusals and preserved capabilities; those are publisher claims, not measured gains for this project.[8] Confirm derivative provenance before use, and compare refusal behavior separately from factual accuracy, correction handling, and authorized action. Train the initial adapter from official safetensors weights; a GGUF serving artifact is not the starting point for this proposed PEFT training workflow.

For a second model family, Ai2's Olmo is especially relevant to the open-source objective because it publishes training data and training procedures as well as weights. Its 7B/32B Olmo 3 family offers a separately developed reference, with artifact-level provenance checks still required.[9] Microsoft's MIT-licensed Phi-4 and IBM's Apache-licensed Granite are additional candidates, not required first-week work.[10][11] Comparing a small number of eligible bases on the actual tasks is more informative than importing a general leaderboard ranking.

Improvement methods

LoRA freezes base weights and learns low-rank updates; QLoRA combines trainable adapters with a quantized frozen base. Their main benefit is making adaptation and distribution less expensive. They do not remove the need to propagate gradients through the network to reach earlier adapters.[12][13]

Method	Appropriate problem	Proposed position
Better retrieval, prompts, schemas, and memory	Missing current evidence or poorly specified behavior	Establish a strong unchanged-weight baseline first.
Supervised LoRA	Repeated, teachable errors in evidence use, corrections, or tool actions	Main initial program.
Preference optimization	Choosing between already plausible responses or actions	Add after collecting reliable preference pairs.
Distillation from an eligible teacher	Transferring successful task behavior to a smaller student	Filter with evidence and executable checks; compare with human-only data.
Continued pretraining or broader weight updates	Persistent domain-capability gaps after data and adapter experiments	Later bounded research track.
Reinforcement learning with verified rewards	Tasks with a dependable external success criterion	Later tool sandbox experiment, after the verifier is audited.

A study comparing LoRA with full fine-tuning on programming and mathematics found that conventional low-rank settings learned less while preserving more out-of-domain performance. This is a useful warning against treating LoRA as universally equivalent to full tuning; it is not a Gemma 4 result.[14] If a carefully evaluated adapter stalls, first inspect examples, supervision, retrieval, rank, and target coverage. Then compare a larger base or limited broader tuning on a smaller dense model.

DPO can learn from preferred/rejected answer pairs without a separate online reinforcement-learning loop.[15] A first preference set could contrast a correct narrow clarification with an invented identity, or a cited qualified conclusion with an unsupported confident claim. Keep response lengths comparable and adjudicate the reason for preference. Do not optimize for agreeableness, number of actions, or absence of refusals as substitutes for useful completion.

Intel training path

Use **native PyTorch XPU + Transformers + PEFT**, with an inspectable supervised training loop or TRL after its masking checks pass. Begin in BF16 with ordinary AdamW on adapter parameters, eager execution or a supported PyTorch attention path, and gradient checkpointing. Avoid making fused CUDA kernels, paged optimizers, or an inference engine part of the training dependency chain.

A current package candidate set is PyTorch 2.14.0, Transformers 5.17.0, PEFT 0.20.0, TRL 1.13.0, Accelerate 1.15.0, and optional bitsandbytes 0.50.2. These are observed published versions, **not a jointly tested lock**. Select the appropriate XPU PyTorch distribution, resolve the environment on a site-compatible host, and retain exact wheel hashes and dependency versions after the probe succeeds.[16]

bitsandbytes' 0.50.0 release matrix lists Max Series support for four-bit LoRA and eight-bit optimizers. Its XPU installation page currently specifies Linux wheels requiring glibc 2.34 and documents oneAPI build versions.[17][18] Inspect the actual OS, glibc, driver, Level Zero runtime, and library resolution on the allocated node. Do not assume that installing a recent wheel also updates the host driver or that a generic Linux support table settles the XPU wheel requirement.

Use dense QLoRA as a measured optimization after BF16 works: NF4 frozen weights, BF16 computation, matching data and adapter settings. Compare both task quality and end-to-end training cost; lower memory use need not imply lower elapsed time. Audit which tensors were quantized rather than relying on a configuration flag.

For MoE, Transformers' Gemma implementation stores routed expert matrices as stacked `nn.Parameter` tensors. The shared dense MLP and router are distinct. Consequently, ordinary `target_modules` names for dense MLP projections do not mean that routed experts have been adapted.[19] PEFT documents direct targeting through `target_parameters`, including shape constraints and extra runtime costs.[20]

Axolotl's expert quantization implementation explicitly requires CUDA and its optimized expert paths are not evidence of XPU support.[21] Its Gemma guide also documents attention head-size constraints and observed gradient spikes in particular tuning configurations.[22] For the first PVC MoE experiment, keep experts and routing frozen in BF16 and adapt text attention; consider the shared MLP as a separately labeled experiment. Treat expert quantization, expert adapters, and router tuning as later, separate compatibility projects.

Inspect instantiated module paths before creating adapters. Freeze multimodal input components, embeddings, and output heads initially; constrain targets to the text backbone. Gemma variants can share K/V computation or omit expected projections in some layers, so save every matched target, shape, trainable count, and unmatched expectation. Reject a setup that accidentally includes a router or silently adapts no useful parameters.

For supervised training, inspect decoded tokens and loss masks on at least 20 examples. TRL's assistant-only loss requires a compatible template, and completion-only training has a different dataset contract.[23] Ensure that user text, retrieved documents, padding, and fictional tool results receive no target loss unless deliberately part of the objective. Preserve the model's native conversation and tool delimiters; begin with thinking disabled and evaluate thinking-enabled behavior separately.

Use an initial sequence length of 1,024 for debugging, then measure 2,048 and 4,096. Large vocabulary logits and retained activations can dominate memory even when adapter state is small. Adopt packing, compiled attention, or chunked loss only after checking masking, gradients, and numerical behavior against the simpler path. An advertised 256K context window does not make 256K training a sensible starting configuration.

First execution gates

These are proposed acceptance checks for future allocated-node work, not completed measurements.

1. **Environment receipt:** record scheduler allocation, device/tile mapping, visible memory, OS/runtime versions, exact model and tokenizer revisions, and the resolved package lock. Run a small BF16 tensor backward pass and synchronize the device.

2. **Model receipt:** complete forward/loss/backward on fictional text; confirm finite loss and gradients, and that intended adapters change after an optimizer step. Verify sampled frozen base tensors remain unchanged and checkpointing does not disconnect adapter gradients through frozen embeddings.

3. **Learning diagnostic:** fit a deliberately tiny set of 32 fictional examples for a bounded run. Compare loss before and after on that same set to expose broken supervision. This is an implementation diagnostic, not quality evidence.

4. **Recovery receipt:** save adapter, optimizer, scheduler, random states, data cursor, and configuration; reload in a fresh process and resume. Compare with an uninterrupted reference within declared numerical tolerances.

5. **Placement receipt:** measure steady-state processed tokens/second, supervised tokens/second, peak allocated/reserved memory, step-time distribution, and checkpoint/load time. Include warmup separately.

6. **Serving receipt:** load the exact adapter with its base in a verified runtime and compare a fixed output suite before and after export/quantization. Test rollback to the base.

Give a compatibility lane an initial four-node-hour troubleshooting cap. If it does not pass, retain the failing environment and move that lane to diagnosis while passing lanes continue. This prevents expert-kernel investigation from consuming the full model-improvement program.

When sharding is required, prove single-node communication and training before attempting multiple nodes. PyTorch recommends XCCL for XPU distributed training.[24] Data parallelism replicates base weights and is useful for throughput; it does not solve a per-device fit problem. FSDP can address weight placement, but its interaction with frozen parameters, PEFT, checkpointing, and the selected Gemma implementation requires its own test. Prefer independent experiments across nodes until a measured need justifies distributed training.

Data and source management

The first corpus should combine authored fictional workflow examples, project-owned software documentation, and explicitly reusable public evidence. Public availability is not a sufficient dataset license check. PMC's Open Access subset includes varying license terms and provides designated retrieval services; select individual articles under terms compatible with the intended training and redistribution.[25]

Start with approximately **2,000 reviewed training examples**, a separate 200-case development set, a locked 600-case task test set, and a 200-case general regression set. These are initial design sizes, not claims of statistical sufficiency for every subgroup. Expand training toward 10,000 only when reviewed error categories justify it. Track source diversity and template families; thousands of paraphrases of a few examples do not constitute broad evidence.

An initial training mixture can allocate 40% to evidence use, 25% to corrections and ambiguity, 20% to source/sharing/tool workflows, and 15% to general behavior preservation. This is a starting allocation to revise on development results. Train the first shared workflow adapter on that mixture; only create specialist adapters where task differences or measured interference justify them.

Examples should include corrected organization names, patients who are also clinicians or family members, references to similarly named people, and requests that change audience halfway through a conversation. Teach the assistant to incorporate a clear correction, preserve unaffected work, and ask one narrow question when a consequential ambiguity remains. A mistaken name must never itself trigger moving information between accounts or organizations.

The source registry already supplies account ownership, versions, source links, and import receipts. A training layer still needs an explicit dataset snapshot and consent contract. Proposed per-example metadata includes source ID and version, content hash, applicable license or grant, owner/scope, annotation provenance, teacher model revision if any, evidence spans, transformation history, and split-family ID. Hash immutable training exports so later source edits cannot silently change a run's inputs.

Split by document family, scenario template family, project, and participant identity where relevant, before generating paraphrases or teacher responses. Deduplicate across splits. Keep evaluation sources and labels unavailable to training-data generators. An error discovered on the locked test becomes a future benchmark-version decision; it must not become training material while the same test is still used to claim an independent gain.

For synthetic data, an eligible larger Gemma can draft examples only from approved source packets. Retain candidates that pass schema, evidence-span, and task checks, then review a representative sample and all consequential judgments. Use verified answers and concise rationales rather than assuming an unverified reasoning trace is ground truth. Include a human-only versus mixed-data ablation; an extra model vote is not independent evidence.

Evaluation and promotion

The primary endpoint is **accepted task completion rate**: all required factual and action conditions for a scenario are satisfied without a consequential unsupported claim or unauthorized transition. Report every component separately alongside that endpoint. Rate one full scenario, rather than counting every claim from the same response as an independent success.

Locked task set	Cases	Main checks
Evidence use	200	Claims supported by supplied evidence, correct citations/spans, contradictory findings preserved, useful abstention when evidence is missing.
Corrections and ambiguity	100	Corrected entity or intent used consistently; consequential uncertainty resolved with a focused question; unrelated work preserved.
Sharing and privacy	200	Correct account/audience, no private-source exposure, no invented permission, withdrawal and malicious-source scenarios handled.
Tool and source operations	100	Valid arguments, correct source/version, idempotent behavior, sensible recovery from a denied or failed operation.

Evaluate an unchanged model with a competent prompt/retrieval configuration against the adapter using the same evidence packets, tools, output budget, and decoding settings. Add an end-to-end retrieval evaluation separately to locate retrieval failures. Score the actually deployed quantized/exported artifact as well as the training checkpoint. For subjective clarity or evidence interpretation, use blinded human review, with

disagreements adjudicated.

An eligible LLM judge can triage obvious cases after calibration against people. Swap answer order and obscure model identity. Published work documents position, verbosity, and self-preference biases in model judges; judge agreement alone should not decide release.[26] Executable tool state and source evidence should dominate judgments when available.

Proposed promotion gates are: a gain of at least five percentage points in task acceptance over the competent baseline; a paired, scenario-grouped 95% confidence interval excluding zero; no more than a two-point decline on the general regression set; no observed unauthorized disclosure or sharing transition in the locked challenge suite; and an explicit latency/memory budget met by the deployed artifact. Treat uncertain regression estimates as requiring more cases, not as automatic passes. These thresholds are provisional product criteria, not published guarantees.

Select settings on development data, then evaluate the selected configuration across repeated seeds on the locked test. Use grouped, stratified resampling for the paired interval and show raw counts per slice. Zero failures in 200 independent representative trials would still leave an approximate 1.5% one-sided 95% upper bound; a deliberately constructed challenge suite does not justify even that population estimate. Passing the suite never turns a model into an authorization boundary.

Measure useful output per node-hour, total generated and processed tokens, human review minutes per accepted example, completion latency, and correction burden. Logs of “memory used,” “instinct fired,” confidence, or number of agent turns are secondary observations. Evaluate persistent dispositions through behavior across scenarios, including decisions where no explicit event is logged.

After an adapter wins on fixed evidence, cross base/tuned weights with individual memory on/off and shared memory on/off. That eight-cell experiment separates learned behavior from memory access. Keep agent tools and authorized information equivalent within the intended comparison; report experience/token differences rather than assuming equal budgets imply equal learning opportunities.

Reservation schedule and experiment budget

Run hardware bring-up, data/annotation work, and baseline evaluation concurrently. Within the first allocation period, execute a bounded dense BF16 probe, a BF16 MoE attention probe, and an independent baseline-serving lane. A successful lane can begin experiments while a failed lane is repaired. Use CPU resources for parsing, deduplication, deterministic checks, and evaluation orchestration; benchmark any CPU model service before assigning it generation capacity.

The first screening matrix has **two models × two target scopes × two ranks × two seeds = 16 adapter runs**. The models are 12B Unified and 26B-A4B; target scopes are text attention alone and attention plus the dense/shared MLP; ranks are 16 and 64. Keep the routed experts and router frozen in both MoE scopes. If the MoE lane fails compatibility, run the eight dense cells and retain the eight blocked cells explicitly.

Use one epoch over the same frozen training export, initial learning rate `5e-5`, standard LoRA scaling with `alpha = 2r`, dropout `0.05`, microbatch one, and gradient accumulation for a measured effective batch. Begin with conservative gradient clipping and record its frequency; a provisional norm cap of `0.1` is a debugging starting point, not an established optimum for every Gemma. Vary learning rate, epochs, precision, or packing only in subsequent identified comparisons. The experiment worksheet lists the 16 planned cells with empty measurement fields.

For a reservation week, the intended progression is: establish execution and baseline evidence in the opening period; screen adapter settings and curate errors in the next period; replicate selected settings and test dense

QLoRA, data mixtures, and export behavior after that; use the final period for blinded evaluation, peer packaging, and a reproducible report. These are work phases rather than guaranteed calendar completion times. Data review can become the limiting resource even when GPUs are available.

At full Intel reservation capacity, there are 80 physical GPUs. A possible placement is 12 PVC nodes for independent training cells, four for baseline/teacher/evaluation services, two for compatibility and export experiments, and two for retries or longer runs. This is a proposed allocation of 20 nodes, not a claim that every model fits one device or that the current launcher implements it. A separate H100 track can use documented CUDA recipes if included in the reservation execution plan; report its results separately from PVC measurements.

Use node-hours for actual allocation accounting and physical-GPU-hours for utilization; record exposed-tile usage separately. A four-GPU node occupied for two hours costs two node-hours whether one or four GPUs are useful. It offers eight allocated physical-GPU-hours, which must not be reported as eight hours of measured useful training.

For run `i`, estimate `hours_i = setup_i + processed_tokens_i / (3600 × measured_tokens_per_second_i) + checkpoint_and_evaluation_i`. Multiply by occupied nodes, then sum runs; do not multiply again by GPUs when the measured throughput already covers the whole placement. Recalibrate after warmup and after changing sequence length or targets. Separate failed/aborted runs and human annotation time from successful-run throughput.

As an initial scheduling ceiling, 16 screening runs capped at four single-node hours each would reserve at most 64 node-hours, before baselines, repeats, and infrastructure diagnosis. This is a stop budget, not a prediction that an epoch will finish. If several independent runs share one node, count its occupied wall time once. A 20-node, seven-day Intel envelope would be 3,360 node-hours; useful completed experiments depend on measured speed and data readiness, not that arithmetic alone.

TACC describes ordinary queue limits and scheduler behavior; the dedicated reservation's submission mechanism must be recorded when it is available. Use compute allocations for model work, checkpoint to durable run storage, and export elsewhere: TACC documents that `$WORK` is not backed up and `$SCRATCH` is subject to purge.[1] Schedule against the reservation, not by hard-coding particular node names.

Peer distribution and learning permissions

Build the first shared adapters from redistributable public/synthetic data and allow each peer to opt in. A release bundle should include safetensors adapter weights, exact base/tokenizer hashes, target manifest, training configuration, dataset provenance manifest, evaluation results, license notices, and an integrity signature. A peer verifies compatibility and evaluates in a temporary workspace before activating a version; rollback selects its previous adapter or the base.

vLLM documents adapter serving, but general LoRA support does not establish compatibility for every Gemma/XPU/quantization combination.[27] Test live adapters first. If that serving path is unsupported, merge a release copy into the matching base, export to a validated serving format, and rerun the acceptance suite. Keep the unmerged adapter and training receipt so the model remains reproducible and future changes are attributable.

Preserve simple independent choices for each source: **use in this assistant**, **use for local training**, **contribute to shared training**, and **publish the resulting adapter**. Default the three training/distribution choices off. Organizational membership or public visibility should not silently switch them on; the source owner should be able to save and preview those choices before applying them. Shared training requires permission from every included source, with conflicts resolved by the narrower grant.

Mutable personal facts belong in access-controlled retrieval and editable memory. Training on them makes correction and withdrawal harder. Turning sharing off can stop future transfer and retrieval, but it cannot erase information already absorbed into released weights or already downloaded by peers. Plan replacement/retraining from an excluded-source dataset when needed, and disclose the limits of previously distributed copies rather than promising instant unlearning.

Peer-to-peer operation does not require federated training in the first release. Independent peers can obtain the same public adapter while keeping private retrieval and optional local adapters private. A later federated experiment should use fictional/public partitions first, retain exact base compatibility, test malicious updates, and explicitly define coordinator trust and participation rules.

Federated LoRA aggregation is a research issue of its own. Averaging low-rank factors separately does not generally equal averaging the represented weight updates; FLoRA studies stacking-based aggregation to address this problem.[28] Sharing gradients or adapters also does not inherently provide a privacy guarantee: gradient-reconstruction research demonstrates leakage under specific conditions.[29] Secure aggregation and differential privacy would need explicit threat models, accounting, and utility tests before introducing private participant data.

Deliverables and remaining uncertainties

The first milestone is a functioning, recoverable training recipe; an unchanged-weight baseline; an immutable reviewed dataset; and an adapter with a blinded comparison against that baseline. The second is an installable peer artifact with provenance, regression results, and rollback. A negative result is useful if it distinguishes a data problem, base-capability limit, or hardware/software incompatibility.

Implementation work should add a dedicated training package and launcher, dataset snapshot/export and consent metadata, task evaluation fixtures, and an adapter registry. Reuse source receipts and model-policy manifests where appropriate. Keep this work separate from the existing Stampede inference campaign and the running dashboard demo; the current inference launcher is not a fine-tuning launcher.

Unresolved execution facts are the allocated node's actual device/tile presentation, driver and wheel compatibility, Gemma-specific backward and checkpoint behavior, measured model placement, training throughput, and Gemma adapter-serving compatibility on the chosen backend. Unresolved research questions are the best task/data mixture, whether 26B's serving efficiency compensates for its training complexity, whether preference training adds value, and whether a smaller adapter-equipped peer model meets the quality threshold.

The immediate recommendation is therefore **12B BF16 LoRA and 26B BF16 attention LoRA in parallel, evaluated against strong unchanged-model baselines**, with E4B as the smaller deployment path and 31B as a larger reference. Expand the program through measured error correction and replicated experiments; use the whole reservation to compare hypotheses rather than assuming one large training run is the best use of it.

Sources

All live documentation and repository metadata below were accessed September 13, 2026. Dates identify the publication/update where available; undated documentation is identified as such. Versioned sources and checkpoint revisions should be preferred when reproducing the plan.

1. Texas Advanced Computing Center. Stampede3 User Guide. Page labels last update June 2, 2026, with later notices. Architecture, scheduling, and storage sections.

2. Texas Advanced Computing Center. Stampede3 system overview. Undated overview; its 10-node GPU inventory differs from the operational guide.

3. Intel. Gemma 4 Models optimized for Intel Hardware: Enabling instant deployment from day zero. 2026; deployment and upstream framework support.

4. PyTorch contributors. Getting Started on Intel GPU. PyTorch 2.14 documentation; updated August 21, 2026.

5. Google DeepMind. Gemma 4 model card. Updated July 30, 2026.

6. Google. Official Hugging Face model metadata: E4B, 12B, 26B-A4B, and 31B. Live metadata snapshot; exact revisions and parameter totals retained in the companion inventory.

7. Hugging Face contributors. Gemma4 Unified, Transformers 5.17.0; and Google, 12B configuration at revision 707f0a3. Undated versioned documentation/configuration.

8. HauhauCS. Gemma4-26B-A4B-Uncensored-HauhauCS-Balanced model card. Undated release-candidate card; claims attributed to its publisher.

9. Allen Institute for AI. Olmo 3: Charting a path through the model flow to lead open-source AI. 2025; and Olmo model family, live project page.

10. Microsoft Research. Phi-4 model card. Live official model card; MIT license.

11. IBM Granite team. Granite-4.0-H-Small model card. Released October 2, 2025; Apache 2.0 license.

12. Edward J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. 2021.

13. Tim Dettmers et al. QLoRA: Efficient Finetuning of Quantized LLMs. 2023.

14. Dan Biderman et al. LoRA Learns Less and Forgets Less. 2024.

15. Rafael Rafailov et al. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. 2023; Hugging Face, DPO Trainer, live documentation.

16. Package publisher metadata on PyPI: torch, transformers, peft, trl, accelerate, bitsandbytes. Live version metadata; publication does not demonstrate interoperability.

17. bitsandbytes contributors. Release 0.50.0 support matrix. Versioned repository documentation.

18. bitsandbytes contributors. Installation Guide, Intel XPU. Live documentation; wheel requirements can change.

19. Hugging Face contributors. Gemma4 implementation. Transformers 5.17.0; text experts, router, and attention implementation.

20. Hugging Face PEFT contributors. LoRA reference. Live documentation; direct parameter targeting and MoE considerations.

21. Axolotl contributors. MoE Expert Quantization. Live documentation; explicit CUDA requirement.

22. Axolotl contributors. Gemma 4 training guide. Live documentation; architecture limitations and tuning observations, not PVC benchmarks.

23. Hugging Face TRL contributors. SFT Trainer. Live documentation; completion/assistant masking and PEFT integration.

24. PyTorch contributors. Distributed communication package. PyTorch 2.14; XCCL backend guidance.

25. US National Library of Medicine. PMC Open Access Subset. Live documentation; article-level licenses and retrieval methods.

26. Lianmin Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. 2023.

27. vLLM contributors. LoRA Adapters. Live documentation; general serving support, not a validation of the proposed Gemma/XPU combination.

28. Ziyao Wang et al. FLoRA: Federated Fine-Tuning Large Language Models with Heterogeneous Low-Rank Adaptations. 2024.

29. Ligeng Zhu, Zhijian Liu, Song Han. Deep Leakage from Gradients. NeurIPS 2019.